

Домашнее задание № 4

1. Создание тестовой таблицы и наполнение ее тестовыми данными

```
CREATE TABLE transactions (  
  transaction_id UInt32,  
  user_id UInt32,  
  product_id UInt32,  
  quantity UInt8,  
  price Float32,  
  transaction_date Date  
) ENGINE = MergeTree()  
ORDER BY (transaction_id);  
  
-- Заполнение тестовыми данными (100 транзакций)  
INSERT INTO transactions (transaction_id, user_id, product_id, quantity, price, transaction_date)  
SELECT  
  number + 1 AS transaction_id,  
  rand() % 20 + 1 AS user_id, -- 20 уникальных пользователей  
  rand() % 10 + 1 AS product_id, -- 10 продуктов  
  rand() % 5 + 1 AS quantity, -- количество от 1 до 5  
  round((rand() % 100 + 10) / 10.0, 2) AS price, -- цена от 1.0 до 10.0  
  toDate('2026-01-01') + rand() % 30 AS transaction_date -- даты в январе 2026  
FROM numbers(100);  
  
SELECT * FROM transactions;
```

Результат 1 X

SELECT * FROM transactions | Введите SQL выражение чтобы отфильтровать результаты

	123 transaction_id	123 user_id	123 product_id	123 quantity	123 price	123 transaction_date
1	1	12	2	2	2,1	2026-01-02
2	2	7	7	2	9,6	2026-01-07
3	3	6	6	1	3,5	2026-01-06
4	4	9	9	4	1,8	2026-01-29
5	5	15	5	5	2,4	2026-01-25
6	6	2	2	2	9,1	2026-01-12
7	7	4	4	4	7,3	2026-01-14
8	8	6	6	1	1,5	2026-01-16
9	9	15	5	5	6,4	2026-01-05
10	10	3	3	3	9,2	2026-01-13

2. Агрегатные функции

```
-- Все показатели в одном запросе  
SELECT  
  sum(price * quantity) AS total_revenue, -- общий доход от всех операций  
  avg(price * quantity) AS avg_revenue_per_transaction, -- средний доход от одной сделки  
  sum(quantity) AS total_quantity_sold, -- общее количество проданной продукции  
  count(DISTINCT user_id) AS unique_users, -- количество уникальных пользователей  
  count(*) AS total_transactions -- дополнительно: общее количество транзакций  
FROM transactions;
```

Результат 1 X

SELECT sum(price * quantity) AS total_revenue, avg | Введите SQL выражение чтобы отфильтровать результаты

	123 total_revenue	123 avg_revenue_per_transaction	123 total_quantity_sold	123 unique_users	123 total_transactions
1	1 651,4000039101	16,5140000391	295	20	100

3. Функции для работы с типами данных

```

-- Все преобразования в одном запросе
SELECT
  -- Исходные данные
  transaction_id,
  user_id,
  product_id,
  quantity,
  price,
  transaction_date,
  -- Преобразования
  toString(transaction_id) AS transaction_id_str,
  formatDateTime(transaction_date, '%Y-%m-%d') AS date_formatted,
  concat(toYear(transaction_date), '-', toMonth(transaction_date)) AS "year-month",
  round(price) AS price_rounded
FROM transactions
LIMIT 10;

```

	user_id	product_id	quantity	price	transaction_date	transaction_id_str	date_formatted	year-month	price_rounded
1	12	2	2	2,1	2026-01-02	1	2026-01-02	2026-1	2
2	7	7	2	9,6	2026-01-07	2	2026-01-07	2026-1	10
3	6	6	1	3,5	2026-01-06	3	2026-01-06	2026-1	4
4	9	9	4	1,8	2026-01-29	4	2026-01-29	2026-1	2
5	15	5	5	2,4	2026-01-25	5	2026-01-25	2026-1	2
6	2	2	2	9,1	2026-01-12	6	2026-01-12	2026-1	9
7	4	4	4	7,3	2026-01-14	7	2026-01-14	2026-1	7
8	6	6	1	1,5	2026-01-16	8	2026-01-16	2026-1	2
9	15	5	5	6,4	2026-01-05	9	2026-01-05	2026-1	6

4. UDF-функции

```

-- Создание UDF для подсчета общей суммы операции
CREATE FUNCTION transaction_total AS (quantity, price) -> quantity * price;

-- UDF для классификации транзакций по пороговому значению
CREATE FUNCTION classify_transaction AS (amount, threshold) ->
  if(amount >= threshold, 'high_value', 'low_value');

-- Альтернативная версия с более детальной классификацией
CREATE FUNCTION classify_transaction_detailed AS (amount) ->
  multiIf(
    amount >= 100, 'Высокоценная (>100)',
    amount >= 50, 'Среднеценная (50-100)',
    amount >= 20, 'Низкоценная (20-50)',
    'Минимальная (<20)'
  );

```

```

-- Все UDF в одном запросе
SELECT
  transaction_id,
  user_id,
  product_id,
  quantity,
  price,
  transaction_total(quantity, price) AS total_amount,
  classify_transaction(total_amount, 100) AS category_simple,
  classify_transaction_detailed(total_amount) AS category_detailed,
  transaction_date
FROM transactions
ORDER BY total_amount DESC
LIMIT 10;

```

	transaction_id	user_id	product_id	quantity	price	total_amount	category_simple	category_detailed
1	57	20	10	5	10,9	54,4999980927	low_value	Среднеценная (50-100)
2	20	15	5	5	10,4	51,9999980927	low_value	Среднеценная (50-100)
3	36	15	5	5	10,4	51,9999980927	low_value	Среднеценная (50-100)
4	78	5	5	5	9,4	46,9999980927	low_value	Низкоценная (20-50)
5	41	20	10	5	8,9	44,4999980927	low_value	Низкоценная (20-50)
6	83	20	10	5	8,9	44,4999980927	low_value	Низкоценная (20-50)
7	30	9	9	4	9,8	39,2000007629	low_value	Низкоценная (20-50)
8	27	9	9	4	9,8	39,2000007629	low_value	Низкоценная (20-50)
9	19	4	4	4	9,3	37,2000007629	low_value	Низкоценная (20-50)